Chapter 6 | Python Programming

6.1 Python Basics — Interpreter, Comments, Identifiers

Python is an **interpreted**, dynamically typed, case-sensitive language. In interactive mode, **>>> means the INTERPRETER is ready to take the next instruction** — it is the shell prompt, not a shift operator (a continuation line shows three dots …). Blocks are defined by **indentation**, not braces — the colon (:) opens a block and the indented lines belong to it.

(1) **Comments start with #** (asked twice) — they make the code more readable and are ignored by the interpreter. Multi-line commentary is usually written as a triple-quoted string.

(2) **Identifier rules**: an identifier **CAN contain the underscore**; it **cannot be a keyword, cannot start with a digit, and has NO length restriction** — the three wrong options of the same question. (Same family as C's rules — letters, digits, underscore; never digit-first.)

(3) Variables need no declaration — the type is attached to the value (dynamic typing), and the same name may later hold a value of another type.

Exam Points — 6.1

✦ >>> = interpreter ready for the next instruction. # begins a comment (×2). Identifiers: underscore allowed; no keywords, no digit-first, no length limit.

6.2 Data Types and the Four Collections

Core built-in types: int, float, complex, bool, str — and the four collection types, the heart of Python questions:

Collection	Brackets	Ordered?	Mutable?	Duplicates?
list	[ ]	Yes (by position)	**Yes**	Yes
tuple	( )	Yes	**No — immutable**	Yes
set	{ }	No	Yes (elements must be immutable)	**No**
dict	{key: value}	Insertion order (3.7+)	Yes	**Values may repeat; KEYS must be unique**

(1) **Tuple = the sequence that resembles a list but is IMMUTABLE** — once created, elements cannot be added, removed or altered (which is why a tuple can serve as a dictionary key and a list cannot). Parentheses are optional in assignment: course = "l", "m", "n", "o" is a tuple of **four elements**; and although elements cannot change, **del tuple_name deletes the ENTIRE tuple** — both statements true (the source key marking the del statement false is corrected). Printing a mixed tuple shows parentheses with quoted strings: **('Example', 1, 'Tuple')**.

(2) **Dictionary**: organises information by **ASSOCIATION (key), not position**; **values can be repeated but keys must be unique**. Iterating a dict yields its **keys in insertion order** (Python 3.7+), so the worked loop prints "Dictionary Iteration" then **lmn 123** and **abc 345**.

(3) **List methods**: **append() adds an element to the END**; pop() removes (last by default); sort() orders in place; index() finds a position; insert(i, x) places anywhere; remove(x) deletes by value.

Exam Points — 6.2

✦ Tuple = immutable list-like sequence; 4-element unbracketed tuple; del removes the whole tuple (key corrected); prints as ('Example', 1, 'Tuple').

✦ Dict = association by key, not position; keys unique, values repeatable; iteration in insertion order (lmn 123 / abc 345). append() = add at the end.

6.3 Operators and Control Flow

(1) if / elif / else with a colon and indentation. Worked: a = 20, b = 35; a >= b is false → the else branch prints **No**.

(2) Loops: for iterates over any sequence (for i in d walks a dict's keys); while repeats on a condition; range(n) supplies 0 … n−1; break and continue behave as in C.

(3) Operator families mirror C: arithmetic (double-star for power, // for floor division), comparison, logical (and, or, not — words, not symbols), membership (in, not in), identity (is).

Exam Points — 6.3

✦ if a >= b with 20/35 → else → No. Blocks by colon + indentation; logical operators are the words and/or/not.

6.4 Strings and Built-in Functions

Function / method	Asked meaning
strip()	**Returns the string with leading AND trailing white spaces removed** (the other options in the question describe split(), pow() and capitalize())
split()	Returns a **list** cut from the string on the given delimiter
capitalize()	First character upper case, rest lower
max() on a string	**Returns the character with the HIGHEST ASCII (code-point) value**; min() the lowest
len()	**Returns the number of characters** in the string — and the size of any sequence (list, tuple, dict, set)
ord() / chr()	**ord() converts a character to its integer code** (ord('A') = 65); chr() is the inverse (chr(65) = 'A')
int(), float(), str()	The type-casting family — int("5") = 5, str(5) = "5"

Exam Points — 6.4

✦ strip() trims both ends; max(string) = highest-ASCII character; len() counts characters; ord() = character → integer.

6.5 File Handling

(1) **File modes**: r read; w write (truncates); a append; add + for both directions and b for binary. **r+ opens for BOTH reading and writing with the file pointer at the START, without truncating** — the asked mode (w+ truncates first; a+ reads and appends with the pointer at the end).

(2) **Repositioning within a file ('file seek') needs NO actual I/O** — seek() merely moves the file-offset pointer; reading, truncating and deleting all touch the data. tell() reports the current offset.

(3) Extensions such as **.txt, .py and .csv are examples of TEXT files** — human-readable byte streams (binary files need the b modes).

Exam Points — 6.5

✦ r+ = read + write, pointer at the start, no truncation. Seek = repositioning, no I/O involved. .txt/.py/.csv = text files.

Quick Revision — 12 points

1. Python is interpreted, dynamically typed, case-sensitive; blocks by colon + indentation.
2. >>> = the interactive prompt: interpreter ready for the next instruction (continuation = …).
3. # starts a comment (asked twice) — readability, ignored at run time.
4. Identifiers: may contain underscore; never a keyword, never digit-first, no length restriction.
5. Collections: list [] mutable; tuple () immutable; set {} unique elements; dict {k: v}.
6. Tuple = immutable list-like sequence; course = "l","m","n","o" has 4 elements; del deletes the whole tuple (key corrected); prints ('Example', 1, 'Tuple').
7. Dictionary organises by association (key), not position; keys unique, values repeatable; iteration follows insertion order → lmn 123, abc 345.
8. append() adds at the end of a list (pop removes, sort orders, index locates).
9. a = 20, b = 35: if a >= b → else → prints No.
10. strip() removes leading + trailing spaces; split() → list; max(string) → highest-ASCII character; len() → character count; ord('A') = 65, chr(65) = 'A'.
11. File modes: r+ = read/write from the start (no truncation); w+ truncates; a+ appends. Seek (repositioning) needs no actual I/O.
12. .txt, .py, .csv → text files.

DSSSB – TGT COMPUTER SCIENCE (PRACTICE KING) BOOK

Call/Message: 7011121290 Available @ Amazon: https://amzn.in/d/07E04o9B